

NLP Project 4: Sentiment Analysis using BERT and QA Systems with BiDAF

Fatulla Bashirov

Kamal Aghazada

April 2026

Abstract

This report documents Project 4, which covers two main tasks: (1) analysis of a fine-tuned BERT model for sentiment analysis, including case sensitivity and agglutinative language support, and (2) implementation and evaluation of a Bidirectional Attention Flow (BiDAF) reading comprehension system with both traditional GloVe embeddings and frozen BERT contextual embeddings. We train and evaluate both BiDAF variants on a 5 000-example subset of SQuAD v1.1, reporting Exact Match and F1 metrics. An interactive web dashboard is provided as the extra task.

1 Team Member Contributions

- **Fatulla Bashirov**: implemented and analyzed **Task 1** (BERT sentiment analysis, case sensitivity experiments, Azerbaijani language evaluation) and the **Extra Task** (unified FastAPI dashboard with Docker deployment).
- **Kamal Aghazada**: implemented and analyzed **Task 2** (BiDAF architecture, BERT integration, SQuAD training and evaluation pipeline).

Both team members collaborated on report writing, result validation, and presentation preparation.

2 Motivation

Sentiment analysis and reading comprehension are two fundamental NLP tasks that benefit from pre-trained language models. This project investigates: (1) how a multilingual BERT model handles sentiment classification across languages, particularly for agglutinative languages like Azerbaijani, and (2) whether contextual BERT embeddings improve a BiDAF reading comprehension model compared to traditional static word embeddings.

3 Task 1: Sentiment Analysis using BERT

3.1 Method

We analyzed the pre-trained `nlptown/bert-base-multilingual-uncased-sentiment` model, which is fine-tuned on product reviews in six languages (English, Dutch, German, French, Spanish, Italian) for 5-class star-rating prediction. The model is based on `bert-base-multilingual-uncased` (mBERT) with 167M parameters, 12 attention heads, 12 hidden layers, and a hidden size of 768.

The analysis script (`task1_sentiment_bert.py`) loads the model and systematically answers five questions through programmatic experiments, saving all results to JSON.

3.2 Inputs and Outputs

- **Input**: raw text string, tokenized into `input_ids` (token indices), `token_type_ids`, and `attention_mask` tensors. Maximum sequence length is 512 tokens; longer texts are truncated.
- **Output**: logits tensor of shape `(batch_size, 5)`, converted to class probabilities via softmax. The 5 classes represent star ratings from 1-star (very negative) to 5-stars (very positive).

3.3 Case Sensitivity

Since this is an *uncased* model, all input text is lowercased before tokenization. We verified this empirically with 5 test pairs (lowercase vs. uppercase), confirming that tokenization, predictions, and confidence scores are identical in all cases. Case does not affect accuracy.

3.4 Azerbaijani Language Support

Azerbaijani is an agglutinative Turkic language present in mBERT's vocabulary but absent from the model's fine-tuning data. We tested 5 Azerbaijani–English sentence pairs:

Table 1: Azerbaijani vs English sentiment predictions

Expected	AZ Pred	EN Pred	AZ Conf	EN Conf
Positive	5 stars	5 stars	51.7%	82.5%
Negative	2 stars	1 star	29.5%	69.1%
Neutral	2 stars	3 stars	41.0%	52.5%
Positive	5 stars	5 stars	89.8%	88.4%
Negative	1 star	1 star	43.6%	88.2%

Key findings: the model *can* process Azerbaijani text, but confidence is consistently lower (avg. 51.1% vs. 76.1% for English). Agglutinative words suffer heavy subword fragmentation—e.g., “m^ehsullarⁱmⁱzdan” (“from our products”) is split into 8 subword tokens, losing morphological coherence. For production use, fine-tuning on Azerbaijani data or using a Turkic-specific model is recommended.

4 Task 2: Reading Comprehension with BiDAF

4.1 BiDAF Architecture (Task 2.1)

We implemented the Bidirectional Attention Flow model [1] in PyTorch (`bidaf_model.py`). The architecture consists of seven layers:

1. **Character Embedding:** CNN over character sequences (8-dim embeddings, 100 filters, kernel size 5) with max-pooling.
2. **Word Embedding:** Pre-trained GloVe 6B 100d vectors (frozen during training).
3. **Highway Network:** 2-layer gated combination of character and word embeddings.
4. **Contextual Embedding:** Bidirectional LSTM (hidden dim 100, output dim 200).
5. **Attention Flow:** Computes similarity matrix $S_{ij} = \mathbf{w}^\top [\mathbf{h}_i; \mathbf{u}_j; \mathbf{h}_i \circ \mathbf{u}_j]$, then derives context-to-query and query-to-context attention. Output: $\mathbf{G} = [\mathbf{H}; \tilde{\mathbf{U}}; \mathbf{H} \circ \tilde{\mathbf{U}}; \mathbf{H} \circ \tilde{\mathbf{H}}]$.
6. **Modeling Layer:** 2-layer BiLSTM over the attention output.
7. **Output Layer:** Linear projections over $[\mathbf{G}; \mathbf{M}]$ for start logits, and $[\mathbf{G}; \mathbf{M}_2]$ (with an additional BiLSTM) for end logits.

The model takes word and character indices for both context and query, and outputs start/end position logits over the context. Loss is the average of cross-entropy for start and end positions.

4.2 BERT Integration (Task 2.2)

We created a separate BiDAF_BERT model (`bidaf_bert_model.py`) that replaces layers 1–3 (character embedding, word embedding, highway network) with frozen `bert-base-multilingual-uncased` contextual embeddings. The BERT output (768-dim) is projected to 200-dim via a linear layer with ReLU and dropout, then fed into the same contextual BiLSTM, attention flow, modeling, and output layers as the baseline.

This design isolates the embedding source as the only variable, making the comparison between GloVe and BERT embeddings clean and interpretable.

4.3 Training and Evaluation (Task 2.3)

Dataset: SQuAD v1.1 via HuggingFace Datasets, using 5 000 training and 1 000 validation examples.

Evaluation metrics: Exact Match (EM) and token-level F1 score, computed with standard SQuAD normalization (lowercasing, article/punctuation removal).

Training setup: Adam optimizer, learning rate 10^{-3} , gradient clipping at 5.0, ReduceLROnPlateau scheduler (patience 3), 15 epochs. Baseline uses batch size 32; BERT variant uses batch size 16 (due to memory).

Table 2: Model comparison on SQuAD v1.1 validation set

Model	EM (%)	F1 (%)	Trainable Params	Total Params
BiDAF (GloVe + Char CNN)	19.70	30.11	1,615,774	3,085,474
BiDAF-BERT (Frozen mBERT)	16.40	24.00	1,602,802	168,959,218

Table 3: Training progression (selected epochs)

Epoch	Baseline BiDAF				BiDAF-BERT			
	T.Loss	V.Loss	EM	F1	T.Loss	V.Loss	EM	F1
1	4.03	3.88	2.0	6.9	3.63	3.39	6.3	9.6
5	2.83	3.07	13.9	21.2	2.76	2.89	12.8	18.3
10	1.95	3.01	18.3	27.7	2.30	2.75	15.7	23.0
15	1.21	3.56	19.7	30.1	2.00	2.69	16.4	24.0

4.4 Analysis of Results

Contrary to the common expectation that BERT embeddings improve downstream task performance, the baseline BiDAF with GloVe embeddings outperformed the BERT variant by 3.3 EM points and 6.1 F1 points. Several factors explain this:

1. **Frozen BERT:** the BERT encoder was completely frozen, meaning its representations were not adapted to the QA task. The 768→200 projection layer is the only learned mapping, which may lose information.
2. **Tokenization mismatch:** BERT uses WordPiece subword tokenization, while the BiDAF attention mechanism was designed for word-level tokens. Subword-level attention may dilute the signal for answer span detection.
3. **Small training set:** with only 5 000 examples, the projection layer and downstream BiDAF components have limited data to learn an effective mapping from BERT space.
4. **Multilingual model:** `bert-base-multilingual-uncased` distributes its capacity across 104 languages, making its English representations weaker than a dedicated English BERT.
5. **Character-level features:** the baseline benefits from character CNN embeddings that capture morphological patterns, which the BERT variant lacks.

Notably, the BERT variant achieves lower validation loss (2.69 vs. 3.56) despite lower EM/F1, suggesting it produces better-calibrated probability distributions but less precise span boundaries. Fine-tuning BERT or using `bert-base-uncased` would likely improve results significantly.

5 Extra Task: Web Dashboard

We built a unified FastAPI dashboard (`extra_task_ui/`) with the same glass-card visual style as previous projects, deployed via Docker Compose. The UI provides:

- **Overview:** summary cards with key metrics from both tasks.
- **Task 1 tab:** model architecture details, case sensitivity test results, Azerbaijani vs. English comparison table, and subword tokenization visualization.
- **Task 2 tab:** model comparison table with deltas, canvas-drawn training curves (validation loss and F1), epoch-by-epoch history with model selector, and hyperparameter panels.

Deployment: `docker compose up --build` from the `project4/` directory serves the dashboard at `http://localhost:8000`.

6 Discussion and Conclusion

This project demonstrates two key findings:

1. **Multilingual BERT for sentiment analysis** works across languages present in its vocabulary, but performance degrades significantly for languages not in the fine-tuning data. Azerbaijani predictions are directionally correct but with much lower confidence, primarily due to aggressive subword fragmentation of agglutinative morphology.
2. **BERT embeddings do not automatically improve BiDAF** when frozen and used with a small training set. The GloVe baseline with character-level features outperforms frozen mBERT, highlighting that architecture-embedding compatibility, fine-tuning strategy, and training data volume all matter more than raw embedding quality.

Future work should explore fine-tuning BERT end-to-end within the BiDAF framework, using an English-specific BERT model, and training on the full SQuAD dataset to better leverage contextual embeddings.

References

- [1] M. Seo, A. Kembhavi, A. Farhadi, and H. Hajishirzi, “Bidirectional Attention Flow for Machine Comprehension,” *arXiv:1611.01603*, 2017.
- [2] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *arXiv:1810.04805*, 2019.

- [3] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ Questions for Machine Comprehension of Text,” *arXiv:1606.05250*, 2016.
- [4] J. Pennington, R. Socher, and C. Manning, “GloVe: Global Vectors for Word Representation,” *EMNLP*, 2014.